



Birzeit University

Faculty of Engineering & Technology

Electrical & Computer Engineering Department

ENCS4320: Applied Cryptography

Course Project

SecureChannel: An Authenticated, Encrypted Communication System

Prepared by:

Mohammad Manasrah ID: 1211407

Suhaib Ammous ID: 1222517

Instructors: Dr. Ahmed Shawahna, Dr. Mohammad Husain

Section: 2,1

Date: Wednesday 24th June, 2026

Abstract

This project presents the design and implementation of a complete secure communication system, **SecureChannel**, built entirely from scratch in pure Python using fundamental cryptographic primitives. Every primitive is implemented directly from its official RFC or FIPS specification, with no use of external or standard-library cryptographic modules. The system enables two parties — a client and a server — to communicate over an insecure TCP network while guaranteeing confidentiality, integrity, and authenticity, with support for authenticated associated data.

The protocol operates in two phases. Phase A establishes a fresh shared secret through an ephemeral Elliptic-Curve Diffie–Hellman exchange on Curve25519 (X25519) and mutually authenticates the two endpoints by each computing $\text{HMAC}(\text{PSK}, \text{transcript})$ over the full handshake transcript; a mismatching tag aborts the connection and defeats man-in-the-middle attacks. The shared secret is then expanded through HKDF into two independent directional session keys and per-direction nonce material. Phase B protects every application message with the ChaCha20-Poly1305 authenticated-encryption-with-associated-data construction, where a plaintext header is bound as associated data and a strictly increasing sequence number provides replay and reordering protection.

All seven primitives — SHA-256, HMAC, HKDF, ChaCha20, Poly1305, the ChaCha20-Poly1305 AEAD composition, and X25519 — are validated against the official test vectors published in FIPS 180-4, RFC 2104, RFC 5869, RFC 8439, and RFC 7748, and an end-to-end integration test confirms correct key agreement and the rejection of replayed, reordered, and tampered traffic. The project demonstrates a practical understanding of how modern cryptographic protocols, of the kind deployed in TLS 1.3, SSH, Signal, and WireGuard, are assembled from primitives and reasoned about in the presence of active attackers.

Contents

Abstract	I
1 Overview	1
2 Protocol Design	1
2.1 Handshake (Phase A)	2
2.2 Key Schedule	3
2.3 Message Format	4
2.4 Nonce Construction	4
2.5 Replay and Tampering Protection	5
3 Implementation Notes	5
3.1 SHA-256 (FIPS 180-4)	5
3.2 HMAC (RFC 2104)	6
3.3 HKDF (RFC 5869)	7
3.4 ChaCha20 (RFC 8439)	8
3.5 Poly1305 (RFC 8439)	8
3.6 ChaCha20-Poly1305 AEAD (RFC 8439 §2.8)	9
3.7 X25519 (RFC 7748)	10
4 Test Vector Validation	11
4.1 Live Client–Server Demonstration	13
5 AI Usage Statement	13
6 Individual Contributions	14
Conclusion	16

List of Figures

1	Phase A handshake message exchange. After exchanging ephemeral public keys, each side independently computes the shared secret and an authentication tag over the transcript, then verifies the peer’s tag before proceeding.	3
2	Key schedule. The Diffie–Hellman shared secret and the PSK are combined by HKDF-Extract into a single pseudorandom key, which is then expanded into two directional keys and the base nonce material.	4
3	On-the-wire record format. The header travels in the clear but is authenticated as associated data, so the Poly1305 tag covers both the header and the ciphertext.	4
4	Output of the test suite showing every primitive passing its official RFC/FIPS test vectors.	12
5	Live client–server session: successful handshake followed by an encrypted, authenticated chat exchange.	13

1 Overview

SecureChannel allows two endpoints — a **client** and a **server** — to exchange messages with three security guarantees: **confidentiality** (an eavesdropper learns nothing about message contents), **integrity** (any modification of a message in transit is detected), and **authenticity** (each party is assured it communicates with the legitimate holder of a shared secret, not an impostor). The system additionally supports **associated data**: a plaintext header that travels in the clear yet is cryptographically bound to the message so it cannot be altered undetected.

The session proceeds in two phases:

- **Phase A — Handshake.** The two parties generate ephemeral X25519 key pairs, exchange public keys, derive a shared secret through Elliptic-Curve Diffie–Hellman, and authenticate the exchange by each computing $\text{HMAC}(\text{PSK}, \text{transcript})$ over the full handshake transcript. A mismatching tag aborts the connection. The shared secret is then expanded through HKDF into independent directional session keys and nonce material.
- **Phase B — Secure Messaging.** Every application message is sealed with the ChaCha20-Poly1305 AEAD construction. A strictly increasing per-direction sequence number, bound into both the nonce and the authenticated header, provides replay and reordering protection.

A defining constraint of the project is that the seven underlying primitives are implemented *from scratch* in pure Python from their official specifications (FIPS 180-4, RFC 2104, RFC 5869, RFC 8439, and RFC 7748). No cryptographic library is used anywhere in the implementation; only a small set of non-cryptographic standard modules (`struct`, `os`, `socket`, `argparse`, `typing`) is permitted. Each primitive is validated against the official test vectors published in its standards document, since a primitive that fails its published vectors cannot be considered correctly implemented.

2 Protocol Design

This section describes the handshake sequence, the key schedule, the message format, the nonce construction, and the mechanisms that defend against replay and tampering.

2.1 Handshake (Phase A)

The handshake establishes a fresh shared secret and authenticates both endpoints. Its steps are:

1. **Ephemeral key generation.** Each party generates an ephemeral X25519 private scalar d from `os.urandom` and computes the corresponding public value $Q = X25519(d, 9)$, where 9 is the standard Curve25519 base point. Using *ephemeral* keys provides forward secrecy: the per-session keys cannot be recovered later even if long-term secrets leak.
2. **Public-key exchange.** The parties exchange their 32-byte public values over the transport. The client sends first and then receives; the server receives first and then sends, which avoids a deadlock while keeping the resulting transcript identical on both sides.
3. **Shared-secret computation.** Each party computes the Diffie–Hellman shared secret $s = X25519(d_{\text{self}}, Q_{\text{peer}})$. By the commutativity of scalar multiplication, both sides obtain the same s . An all-zero result, which a low-order malicious public key would force, is rejected.
4. **Mutual authentication.** Each party computes

$$\tau = \text{HMAC-SHA256}(\text{PSK}, T), \quad T = Q_c \parallel Q_s \parallel \text{id}_c \parallel \text{id}_s \parallel \text{version}, \quad (1)$$

where the transcript T binds both ephemeral public keys, both identities, and the protocol version. Each side transmits its tag and verifies the peer's tag against its own using a constant-time comparison. Because the pre-shared key (PSK) is a long-term secret known only to the two legitimate parties, an active attacker who relays or substitutes ephemeral keys cannot produce a matching tag, so **any mismatch aborts the connection**. This binds the freshly exchanged keys to the two PSK holders and is what prevents a man-in-the-middle attack.

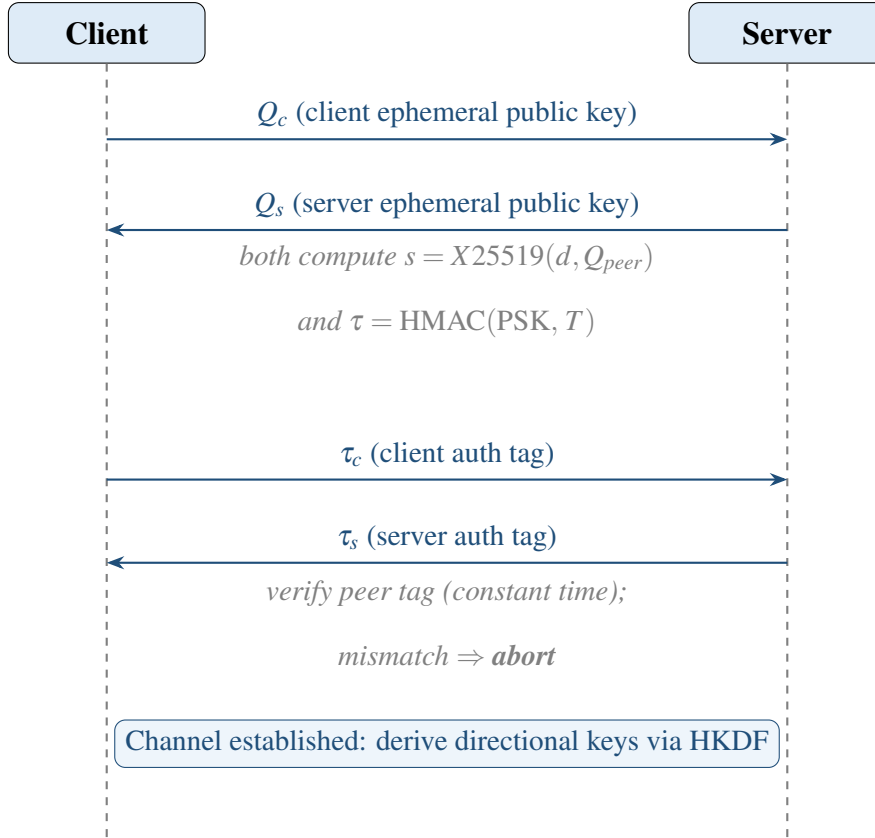


Figure 1: Phase A handshake message exchange. After exchanging ephemeral public keys, each side independently computes the shared secret and an authentication tag over the transcript, then verifies the peer’s tag before proceeding.

2.2 Key Schedule

Once the handshake completes, the shared secret is turned into usable key material with HKDF (RFC 5869), instantiated over our own HMAC-SHA-256. The PSK is used as the salt so that key derivation depends on *both* the Diffie–Hellman secret and the long-term secret:

$$\text{PRK} = \text{HKDF-Extract}(\text{salt} = \text{PSK}, \text{ikm} = s). \quad (2)$$

The pseudorandom key PRK is then expanded into three logically distinct outputs: the client-to-server encryption key, the server-to-client encryption key, and the base nonce material for both directions. Using separate keys per direction means the two data streams never share a keystream, and deriving everything from a single PRK guarantees the outputs are independent pseudorandom strings.

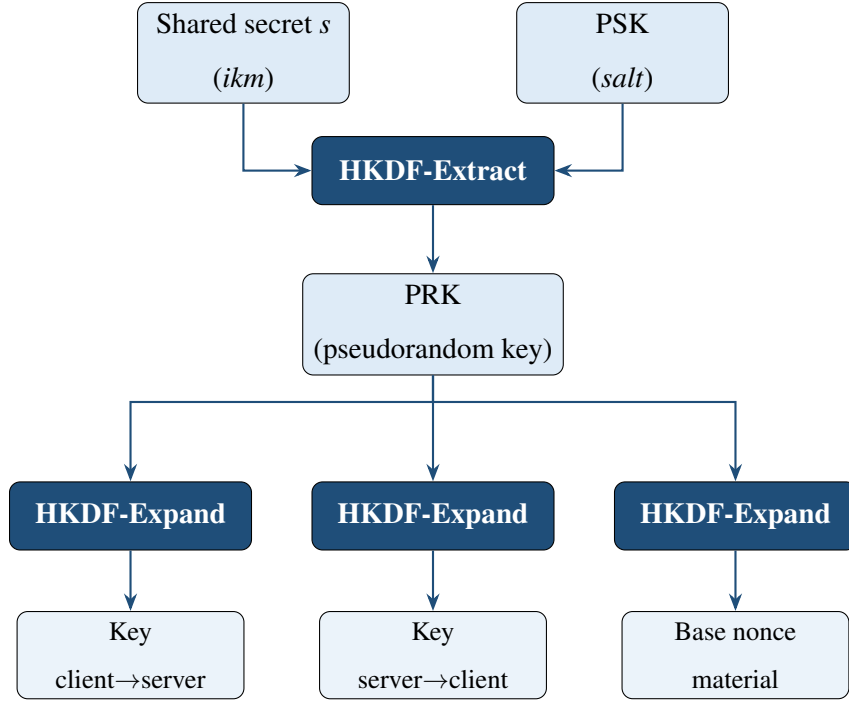


Figure 2: Key schedule. The Diffie–Hellman shared secret and the PSK are combined by HKDF-Extract into a single pseudorandom key, which is then expanded into two directional keys and the base nonce material.

2.3 Message Format

Each Phase B record is a self-describing frame. A 4-byte length prefix delimits the record on the TCP byte stream. Inside, a plaintext *header* acts as associated data: it carries the protocol version, a message type, the sender identity, and a per-direction sequence number. The header is followed by the ChaCha20 ciphertext and the 16-byte Poly1305 authentication tag.

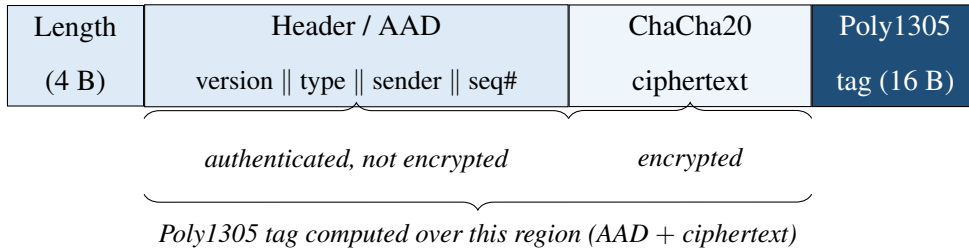


Figure 3: On-the-wire record format. The header travels in the clear but is authenticated as associated data, so the Poly1305 tag covers both the header and the ciphertext.

2.4 Nonce Construction

ChaCha20-Poly1305 requires a unique 96-bit nonce for every message encrypted under a given key; reusing a nonce is catastrophic, as it leaks the XOR of plaintexts and breaks the one-time

MAC. We construct the per-record nonce deterministically from the direction’s base nonce and the sequence number:

$$\text{nonce} = \text{base_nonce} \oplus \text{pad}_{96}(\text{seq\#}), \quad (3)$$

where the 64-bit sequence number is right-aligned and zero-padded to 96 bits. Because each direction has its own base nonce and the sequence number strictly increases, no nonce is ever repeated within a direction, and the two directions use different keys regardless. This makes nonce management stateless and collision-free without transmitting the nonce explicitly.

2.5 Replay and Tampering Protection

Two mechanisms work together. First, each direction maintains a **strictly increasing** sequence number; the receiver records the highest sequence number it has accepted and rejects any record whose sequence number is not strictly greater. This detects and discards replayed or reordered records. Second, the sequence number lives inside the authenticated header, and the Poly1305 tag is computed over both the header (associated data) and the ciphertext. Consequently, an attacker cannot modify the sequence number, sender, type, or any ciphertext byte without invalidating the tag, and the sequence number is committed only after the tag verifies. The combination ensures that out-of-order delivery, replay, header forgery, and ciphertext modification are all detected and rejected.

3 Implementation Notes

Each primitive was implemented directly from its specification. We summarise how each works and the key implementation decisions, with representative excerpts from our source code.

3.1 SHA-256 (FIPS 180-4)

SHA-256 is a Merkle–Damgård hash producing a 256-bit digest. The message is **padded** by appending a single 1 bit (the byte `0x80`), then enough 0 bits so the length is congruent to $448 \bmod 512$, and finally the original message length in bits as a 64-bit big-endian integer; this makes the padded length an exact multiple of the 512-bit block size. Each 512-bit block is processed by the **compression function**: the 16 message words are expanded into a 64-word

message schedule W_t using the small-sigma functions

$$\sigma_0(x) = \text{ROTR}^7(x) \oplus \text{ROTR}^{18}(x) \oplus \text{SHR}^3(x), \quad \sigma_1(x) = \text{ROTR}^{17}(x) \oplus \text{ROTR}^{19}(x) \oplus \text{SHR}^{10}(x). \quad (4)$$

Eight **working variables** $a, \dots, h$ are initialised from the current hash state and updated over 64 rounds using the big-sigma functions, the choice (Ch) and majority (Maj) functions, the per-round **constants** K_t (the fractional parts of the cube roots of the first 64 primes), and the schedule word W_t . After the rounds, the working variables are added back into the hash state modulo 2^{32} .

Listing 1: SHA-256 compression round (64 iterations)

```

1 for t in range(64):
2     big_s1 = _rotr(e, 6) ^ _rotr(e, 11) ^ _rotr(e, 25)
3     ch = (e & f) ^ ((~e & _MASK32) & g)
4     t1 = (h + big_s1 + ch + _K[t] + w[t]) & _MASK32
5     big_s0 = _rotr(a, 2) ^ _rotr(a, 13) ^ _rotr(a, 22)
6     maj = (a & b) ^ (a & c) ^ (b & c)
7     t2 = (big_s0 + maj) & _MASK32
8     h = g; g = f; f = e; e = (d + t1) & _MASK32
9     d = c; c = b; b = a; a = (t1 + t2) & _MASK32

```

Our implementation buffers input so it supports incremental `update()` calls, and computes the digest on a clone of the state so hashing can continue afterwards. All arithmetic is masked to 32 bits.

3.2 HMAC (RFC 2104)

HMAC turns a hash function into a keyed message authentication code:

$$\text{HMAC}(K, m) = H((K_0 \oplus \text{opad}) \parallel H((K_0 \oplus \text{ipad}) \parallel m)). \quad (5)$$

The key is first normalised to one hash block (64 bytes for SHA-256): keys longer than a block are hashed, and shorter keys are zero-padded. The **inner pad** `ipad` is the byte `0x36` repeated and the **outer pad** `opad` is `0x5c` repeated. The **double hashing** structure is essential: the inner hash binds the key to the message, and the outer hash over the result prevents length-extension

attacks that would otherwise apply to a naive $H(K||m)$ construction.

Listing 2: HMAC-SHA-256 core

```
1 def hmac_sha256(key: bytes, message: bytes) -> bytes:
2     k0 = _normalize_key(bytes(key))          # reduce/pad key to one
        block
3     i_key_pad = bytes(b ^ 0x36 for b in k0)  # K0 XOR ipad
4     o_key_pad = bytes(b ^ 0x5C for b in k0)  # K0 XOR opad
5     inner = SHA256(i_key_pad + bytes(message)).digest()
6     return SHA256(o_key_pad + inner).digest()
```

We also implement a constant-time `compare_digest` for tag verification to avoid timing side channels.

3.3 HKDF (RFC 5869)

HKDF follows the **extract-then-expand** paradigm. *Extract* concentrates possibly non-uniform input keying material into a fixed-length pseudorandom key: $\text{PRK} = \text{HMAC}(\text{salt}, \text{ikm})$. *Expand* stretches the PRK into output keying material of the requested length by chaining HMAC calls:

$$T(0) = \varepsilon, \quad T(i) = \text{HMAC}(\text{PRK}, T(i-1) \parallel \text{info} \parallel \text{byte}(i)), \quad (6)$$

and the output is $T(1) \parallel T(2) \parallel \dots$ truncated to the desired length. Feeding the previous block $T(i-1)$ back in chains the blocks together, and appending the single counter byte i ensures each block is distinct.

Listing 3: HKDF-Expand

```
1 def hkdf_expand(prk: bytes, info: bytes, length: int) -> bytes:
2     t = b""; okm = b""; counter = 1
3     while len(okm) < length:
4         t = hmac_sha256(prk, t + info + bytes([counter]))
5         okm += t
6         counter += 1
7     return okm[:length]
```

The maximum output is 255 times the hash length, and an empty salt is replaced by a string of zero bytes, per the specification.

3.4 ChaCha20 (RFC 8439)

ChaCha20 is a stream cipher built from a 4×4 matrix of 32-bit words: four constants ("expand 32-byte k"), eight key words, one 32-bit block counter, and three nonce words. The core is the **quarter-round** $QR(a, b, c, d)$, which mixes four words through additions, XORs, and rotations. A block is produced by applying **20 rounds** — ten iterations of four “column” quarter-rounds followed by four “diagonal” quarter-rounds — to a copy of the state, then adding the original state back word-wise modulo 2^{32} and serialising little-endian. This 64-byte output is the **keystream** block for the current counter value; the cipher encrypts by XORing the keystream with the plaintext and increments the **block counter** for each successive 64-byte block. Decryption is identical to encryption.

Listing 4: ChaCha20 quarter-round

```
1 def _quarter_round(s, a, b, c, d):
2     s[a] = (s[a] + s[b]) & _MASK32; s[d] ^= s[a]; s[d] = _rotl32(s[d]
3         ], 16)
4     s[c] = (s[c] + s[d]) & _MASK32; s[b] ^= s[c]; s[b] = _rotl32(s[b]
5         ], 12)
6     s[a] = (s[a] + s[b]) & _MASK32; s[d] ^= s[a]; s[d] = _rotl32(s[d]
7         ], 8)
8     s[c] = (s[c] + s[d]) & _MASK32; s[b] ^= s[c]; s[b] = _rotl32(s[b]
9         ], 7)
```

3.5 Poly1305 (RFC 8439)

Poly1305 is a **one-time** message authentication code. Its 32-byte key splits into r (the first 16 bytes, partially cleared by a fixed “clamping” mask) and s (the last 16 bytes). The message is divided into 16-byte blocks; each block is read as a little-endian integer with an extra high 1 bit appended, and the accumulator is updated by **polynomial evaluation modulo the prime** $p = 2^{130} - 5$:

$$\text{acc} \leftarrow (\text{acc} + n_i) \cdot r \pmod{p}. \quad (7)$$

After all blocks, s is added modulo 2^{128} and the low 128 bits are serialised little-endian as the tag. The **one-time key requirement** is critical: each (r, s) pair must authenticate at most one message, because reusing it across messages would let an attacker solve for r and forge tags.

Listing 5: Poly1305 accumulation loop

```

1 r = _le_bytes_to_int(key[0:16]) & _CLAMP
2 s = _le_bytes_to_int(key[16:32])
3 acc = 0
4 for offset in range(0, len(message), 16):
5     block = message[offset:offset + 16]
6     n = _le_bytes_to_int(block) + (1 << (8 * len(block))) # append
        high 1 bit
7     acc = ((acc + n) * r) % _P # p =
        2^130 - 5
8 acc = (acc + s) % _2_128
9 return acc.to_bytes(16, "little")

```

3.6 ChaCha20-Poly1305 AEAD (RFC 8439 §2.8)

The AEAD construction **composes** the stream cipher and the MAC in an encrypt-then-MAC arrangement. First a one-time Poly1305 key is generated by running the ChaCha20 block function with the block counter set to 0 and taking the first 32 bytes of keystream. The plaintext is then encrypted with ChaCha20 starting at counter 1. Finally Poly1305 authenticates a precisely formatted message consisting of the associated data, zero padding to a 16-byte boundary, the ciphertext, zero padding to a 16-byte boundary, and the 64-bit little-endian lengths of the associated data and ciphertext.

Listing 6: AEAD seal (encrypt-then-MAC)

```

1 def encrypt(key, nonce, plaintext, aad=b""):
2     otk = _poly1305_key_gen(key, nonce) # ChaCha20
        block, ctr=0
3     ciphertext = chacha20_encrypt(key, 1, nonce, plaintext) # ctr
        starts at 1
4     tag = poly1305_mac(_build_mac_data(aad, ciphertext), otk)
5     return ciphertext + tag

```

On decryption the tag is recomputed over the received associated data and ciphertext and compared in constant time; **only if it matches** is the ciphertext decrypted and the plaintext

returned, so any tampering causes an authentication failure with nothing released.

3.7 X25519 (RFC 7748)

X25519 performs Diffie–Hellman on Curve25519 over the prime field $\mathbb{F}_p$ with $p = 2^{255} - 19$. The scalar is **clamped** during decoding — the lowest three bits of the first byte are cleared, the top bit of the last byte is cleared, and the second-highest bit is set — which forces the scalar into a safe range, ensures it is a multiple of the cofactor, and guarantees a fixed bit length for constant-time operation. Scalar multiplication uses the **Montgomery ladder**, which processes the scalar bit by bit from the most significant bit, maintaining two projective points and performing one differential addition and one doubling per bit. A constant-time conditional swap (`cswap`) selects which of the two working points is updated based on the current scalar bit, so the same operations run regardless of the secret bits.

Listing 7: X25519 Montgomery ladder (per-bit step)

```

1 for t in range(_BITS - 1, -1, -1):
2     k_t = (k_int >> t) & 1
3     swap ^= k_t
4     x2, x3 = _cswap(swap, x2, x3)
5     z2, z3 = _cswap(swap, z2, z3)
6     swap = k_t
7     a = (x2 + z2) % _P; aa = (a * a) % _P
8     b = (x2 - z2) % _P; bb = (b * b) % _P
9     e = (aa - bb) % _P
10    c = (x3 + z3) % _P; d = (x3 - z3) % _P
11    da = (d * a) % _P; cb = (c * b) % _P
12    x3 = (((da + cb) % _P) ** 2) % _P
13    z3 = (x1 * (((da - cb) % _P) ** 2)) % _P
14    x2 = (aa * bb) % _P
15    z2 = (e * ((aa + (_A24 * e) % _P) % _P)) % _P

```

The **field arithmetic** is performed with Python’s arbitrary-precision integers; the final projective coordinate is converted to affine form by a modular inversion computed via Fermat’s little theorem, $z^{-1} \equiv z^{p-2} \pmod{p}$. Public-key generation is X25519 of the private scalar with the base point $u = 9$.

4 Test Vector Validation

Each primitive is validated against the official test vectors published in its RFC or FIPS document, implemented as a dedicated test module under `tests/`. Where the standard library offers a reference implementation, the test files additionally cross-check a batch of random inputs against it — permitted because the project rules forbid cryptographic libraries only inside the implementation, not inside the test files. A primitive that fails its published vectors is regarded as not implemented; all primitives pass, as summarised in Table 1.

Primitive	Source of test vectors	Status
SHA-256	FIPS 180-4 example digests ("abc", empty string, 56-byte and one-million-character messages)	Pass
HMAC-SHA-256	RFC 4231, test cases 1–4, 6, 7	Pass
HKDF-SHA-256	RFC 5869, Appendix A test cases 1–3	Pass
ChaCha20	RFC 8439, §2.3.2 block function and §2.4.2 encryption vectors	Pass
Poly1305	RFC 8439, §2.5.2 and Appendix A.3 vectors	Pass
ChaCha20-Poly1305 AEAD	RFC 8439, §2.8.2 vector, plus tamper-rejection checks	Pass
X25519	RFC 7748, §5.2 and §6.1 vectors, plus iterated tests	Pass

Table 1: Validation of each primitive against its official test vectors.

The complete suite is executed with a single command, and Figure 4 shows the result of the full run.

```
PS C:\Users\Mohammad\Desktop\Crypto> python -m unittest discover -s tests -p "test_*.py" -v
test_block_function (test_chacha20.TestChaCha20.test_block_function) ... ok
test_encryption (test_chacha20.TestChaCha20.test_encryption) ... ok
test_round_trip (test_chacha20.TestChaCha20.test_round_trip) ... ok
test_aad_tamper_rejected (test_chacha20_poly1305.TestChaCha20Poly1305.test_aad_tamper_rejected) ... ok
test_ciphertext_tamper_rejected (test_chacha20_poly1305.TestChaCha20Poly1305.test_ciphertext_tamper_rejected) ... ok
test_decrypt_round_trip (test_chacha20_poly1305.TestChaCha20Poly1305.test_decrypt_round_trip) ... ok
test_encrypt_vector (test_chacha20_poly1305.TestChaCha20Poly1305.test_encrypt_vector) ... ok
test_random_round_trips (test_chacha20_poly1305.TestChaCha20Poly1305.test_random_round_trips) ... ok
test_case_1 (test_hkdf.TestHKDF.test_case_1) ... ok
test_case_2_long (test_hkdf.TestHKDF.test_case_2_long) ... ok
test_case_3_empty_salt_info (test_hkdf.TestHKDF.test_case_3_empty_salt_info) ... ok
test_compare_digest (test_hmac.TestHMAC.test_compare_digest) ... ok
test_cross_check_stdlib (test_hmac.TestHMAC.test_cross_check_stdlib) ... ok
test_rfc4231_vectors (test_hmac.TestHMAC.test_rfc4231_vectors) ... ok
test_appendix_a3_vector_1 (test_poly1305.TestPoly1305.test_appendix_a3_vector_1) ... ok
test_appendix_a3_vector_2 (test_poly1305.TestPoly1305.test_appendix_a3_vector_2) ... ok
test_appendix_a3_vector_3 (test_poly1305.TestPoly1305.test_appendix_a3_vector_3) ... ok
test_rfc8439_2_5_2 (test_poly1305.TestPoly1305.test_rfc8439_2_5_2) ... ok
test_cross_check_hashlib (test_sha256.TestSHA256.test_cross_check_hashlib) ... ok
test_incremental_matches_oneshot (test_sha256.TestSHA256.test_incremental_matches_oneshot) ... ok
test_long_message (test_sha256.TestSHA256.test_long_message) ... ok
test_official_vectors (test_sha256.TestSHA256.test_official_vectors) ... ok
test_iterated_one_round (test_x25519.TestX25519.test_iterated_one_round) ... ok
test_iterated_thousand_rounds (test_x25519.TestX25519.test_iterated_thousand_rounds) ... ok
test_section_5_2_vectors (test_x25519.TestX25519.test_section_5_2_vectors) ... ok
test_section_6_1_diffie_hellman (test_x25519.TestX25519.test_section_6_1_diffie_hellman) ... ok

-----
Ran 26 tests in 5.569s

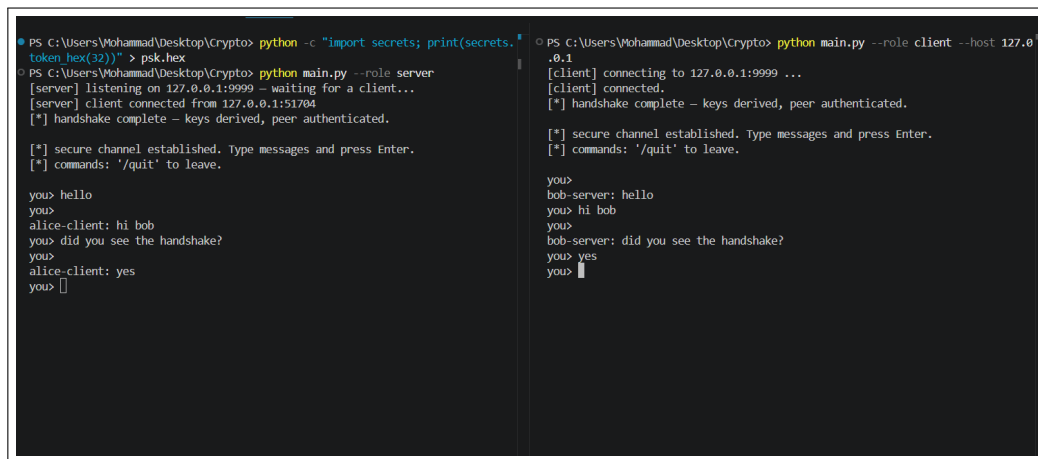
OK
```

Figure 4: Output of the test suite showing every primitive passing its official RFC/FIPS test vectors.

Beyond the per-primitive vectors, an end-to-end integration check exercises the full protocol over a loopback socket: it confirms that both peers derive identical key material, that messages decrypt correctly in both directions, and that replayed records, out-of-order records, ciphertext modifications, header modifications, and handshakes performed under a mismatched PSK are all rejected.

4.1 Live Client–Server Demonstration

The system was also exercised as two live processes communicating over TCP. After the handshake completes, both endpoints report that keys were derived and the peer was authenticated, and they exchange interactive chat messages sealed with the session keys. Figure 5 shows a sample session.



```
PS C:\Users\Mohammad\Desktop\Crypto> python -c "import secrets; print(secrets.token_hex(32))" > psk.hex
PS C:\Users\Mohammad\Desktop\Crypto> python main.py --role server
[server] listening on 127.0.0.1:9999 - waiting for a client...
[server] client connected from 127.0.0.1:51704
[*] handshake complete - keys derived, peer authenticated.
[*] secure channel established. Type messages and press Enter.
[*] commands: '/quit' to leave.

you> hello
you>
alice-client: hi bob
you> did you see the handshake?
you>
alice-client: yes
you>

PS C:\Users\Mohammad\Desktop\Crypto> python main.py --role client --host 127.0.0.1
[client] connecting to 127.0.0.1:9999 ...
[client] connected.
[*] handshake complete - keys derived, peer authenticated.
[*] secure channel established. Type messages and press Enter.
[*] commands: '/quit' to leave.

you>
bob-server: hello
you> hi bob
you>
bob-server: did you see the handshake?
you> yes
you>
```

Figure 5: Live client–server session: successful handshake followed by an encrypted, authenticated chat exchange.

5 AI Usage Statement

In accordance with the course’s AI usage policy (Section 7 of the project specification), this section discloses all AI tool usage by the team.

Tools used: Claude (Anthropic).

How they were used:

- Debugging logic errors in our own primitive implementations (e.g., off-by-one errors in padding, incorrect loop bounds) by describing the symptom and our existing code and asking for help locating the bug.
- Understanding error messages encountered while testing, e.g., Python tracebacks and socket exceptions.
- Looking up Python syntax and standard library usage (e.g., `struct.pack` formatting, `argparse` configuration, socket API usage) for non-cryptographic boilerplate.

- Clarifying concepts from the RFCs that we found difficult to parse on first read — e.g., asking how the Montgomery ladder in RFC 7748 relates to the pseudocode, and why Poly1305 keys must be used only once — in the manner of consulting a tutor, before writing our own implementation independently.
- Generating non-cryptographic boilerplate: TCP socket handling scaffolding, command-line argument parsing structure, and the project’s directory layout.
- Getting feedback on grammar and clarity of this report and the README.

What was not done with AI assistance: The core cryptographic primitives (SHA-256, HMAC, HKDF, ChaCha20, Poly1305, ChaCha20-Poly1305, X25519) and the protocol logic (handshake, key schedule, AEAD channel, replay protection) were designed and written by the team members ourselves, directly from the RFC/FIPS specifications. The analysis, design rationale, and conclusions in this report reflect our own understanding, verified independently by re-deriving each step against the specification before writing it down.

6 Individual Contributions

The work was divided evenly between the two team members, with the two authors pair-programming the shared core (SHA-256) and otherwise taking ownership of distinct primitives, the protocol layer, and the test suite.

Mohammad Manasrah (1211407):

- Co-implemented SHA-256 (`primitives/sha256.py`) with Suhaib.
- Implemented HMAC (`primitives/hmac.py`).
- Implemented the ChaCha20-Poly1305 AEAD composition (`primitives/chacha20_poly1305.py`).
- Implemented X25519 key exchange (`primitives/x25519.py`).
- Wrote `test_sha256.py`, `test_hmac.py`, `test_chacha20_poly1305.py`, and `test_x25519.py` with official RFC/FIPS test vectors.
- Implemented `main.py` (entry point, CLI argument handling, role dispatch).
- Co-authored `README.md` and this report with Suhaib.

Suhaib Ammous (1222517):

- Co-implemented SHA-256 (`primitives/sha256.py`) with Mohammad.
- Implemented the ChaCha20 stream cipher (`primitives/chacha20.py`).
- Implemented the Poly1305 MAC (`primitives/poly1305.py`).
- Implemented HKDF extract and expand (`primitives/hkdf.py`).
- Implemented the protocol layer: `handshake.py`, `session.py`, and `transport.py`.
- Wrote `test_chacha20.py`, `test_poly1305.py`, and `test_hkdf.py` with official RFC/FIPS test vectors.
- Co-authored `README.md` and this report with Mohammad.

Conclusion

This project successfully demonstrates the design and implementation of a complete secure communication system assembled from cryptographic primitives built entirely from scratch. The handshake establishes a fresh, authenticated shared secret using ephemeral X25519 key exchange and a pre-shared-key HMAC over the transcript, defeating man-in-the-middle attacks, while HKDF derives independent directional keys with forward secrecy. The messaging phase protects every record with ChaCha20-Poly1305 authenticated encryption, binds a plaintext header as associated data, and enforces strictly increasing sequence numbers to reject replayed and reordered traffic. Every primitive was validated against its official RFC or FIPS test vectors, and an end-to-end integration test confirmed correct key agreement and the rejection of tampered and replayed messages. Although the system is an educational implementation in pure Python, its architecture mirrors the design of production protocols such as TLS 1.3, SSH, Signal, and WireGuard, providing concrete insight into how modern secure channels are engineered and reasoned about.